

Chapter 1

RTL Design & Implementation of Linear Interpolator (Data Smoother)

1.1 IP Block 3: Linear Interpolator (Data Smoother)

In mixed-signal systems, directly feeding low-sample-rate digital data into a Digital-to-Analog Converter (DAC) produces a "stair-step" waveform known as the Zero-Order Hold (ZOH) effect. This abrupt physical transition introduces severe high-frequency harmonic distortion (aliasing) into the analog domain.

To mitigate this, a hardware **Linear Interpolator** (Data Smoother) was developed as the third IP block. This module acts as a digital front-end filter, artificially increasing the sample rate (upsampling) by mathematically calculating and inserting intermediate data points between two known slow samples, transforming harsh steps into continuous slopes.

1.1.1 Architectural Theory & Hardware Advantages

The module is designed for a fixed $4\times$ upsampling factor, operating on 8-bit signed digital data. Given a previous sample Y_0 and a newly arriving sample Y_1 , the hardware calculates the fractional linear distance using the equation:

$$y(t) = Y_0 + \left(\frac{Y_1 - Y_0}{4} \right) \times i \quad \text{for } i \in \{0, 1, 2, 3\}$$

This approach offers significant improvements in Power, Performance, and Area (PPA) metrics over traditional signal processing methods:

- **Combinational Divider Elimination:** To ensure maximum efficiency, the division by 4 is not implemented using a heavy combinational logic array. Instead, it is executed via a highly efficient **Arithmetic Shift Right** operation ($>>> 2$).
- **Zero-Cost Routing:** In silicon, the arithmetic shift costs zero logic gates as it simply rewires the digital routing two positions over, while preserving the sign bit necessary for negative slope calculations.
- **Synchronous Accumulation:** An internal phase accumulator tracks the fractional steps, outputting a new, mathematically smoothed data point on every single tick of the fast master clock without combinational delay.

1.1.2 Algorithmic Execution (Dual-Rate Pipeline)

During operation, the target data arrives slowly (indicated by the `valid_in` strobe), while the core mathematical accumulator runs on a fast master clock (`clk_fast`) that is $4\times$ faster.

1. **The Target Phase:** When a new sample arrives, the hardware instantly forces the output to the exact value of the old target to prevent rounding error accumulation. It then calculates the new fractional step size.
2. **The Accumulation Phase:** For the next three clock cycles, the hardware simply adds the calculated fractional step to the current output, creating a continuous mathematical ramp towards the new target.

1.1.3 Complete Synthesizable Verilog RTL Description

The following synthesizable RTL implements the $4\times$ interpolator using synchronous pipeline registers and shift-based arithmetic. Bus widths are explicitly declared for direct eSim NGVeri compatibility.

```

`timescale 1ns / 1ps

module linear_interpolator (
    input wire          clk_fast, // High-speed master clock
    input wire          rst_n,    // Active-low asynchronous reset
    input wire          valid_in, // Pulses HIGH every 4th clock cycle
    input wire signed [7:0] data_in, // New low-sample-rate data point

    output reg signed [7:0] data_out, // Smoothed high-sample-rate output
    output reg              valid_out // Output valid strobe
);

    reg signed [7:0] y1; // Target data point buffer
    reg signed [7:0] step_val; // Computed fractional step distance

    always @(posedge clk_fast or negedge rst_n) begin
        if (!rst_n) begin
            y1          <= 8'd0;
            step_val    <= 8'd0;
            data_out     <= 8'd0;
            valid_out    <= 1'b0;
        end else begin
            if (valid_in) begin
                // Snap to old target to prevent rounding accumulation
                data_out <= y1;
                y1 <= data_in;

                // Calculate step: (New Target - Old Target) / 4
                step_val <= (data_in - y1) >>> 2;
                valid_out <= 1'b1;
            end else begin
                // Smoothly accumulate towards the target
                data_out <= data_out + step_val;
                valid_out <= 1'b1;
            end
        end
    end
endmodule

```

Chapter 2

Verification, Co-Simulation & Waveform Analysis

2.1 IP Block 3: Linear Interpolator Verification

2.1.1 System Integration & Schematic Setup

To verify the physical smoothing capabilities of the Linear Interpolator, the RTL was compiled via NGVeri and instantiated within the eSim mixed-signal environment. Crucially, the 8-bit digital output bus (`data_out[7:0]`) was routed directly into an 8-bit Digital-to-Analog (DAC) bridge, allowing the simulator to mathematically sum the bits and plot the final interpolated output as a continuous analog voltage waveform.

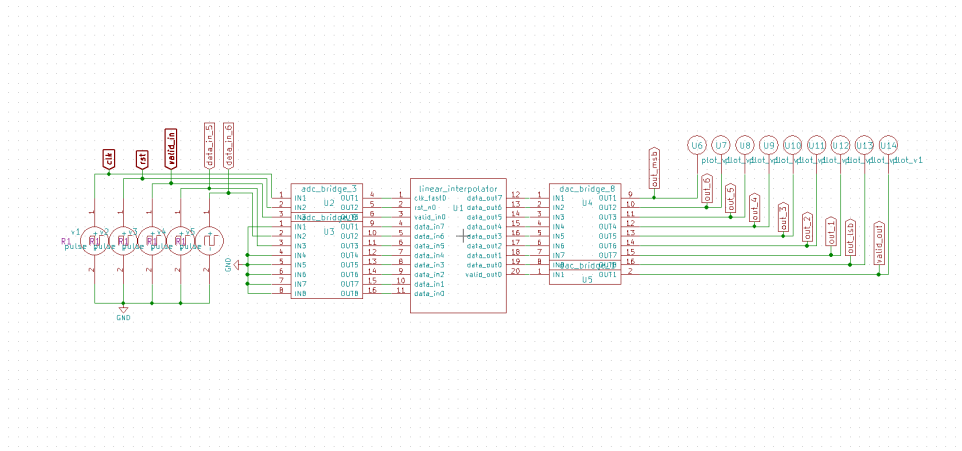


Figure 2.1: eSim Mixed-Signal Schematic of the Linear Interpolator with DAC Boundary

2.1.2 Digital Simulation (Icarus Verilog & GTKWave)

Before mixed-signal translation, the arithmetic accuracy of the linear interpolation pipeline was verified natively using a self-checking testbench in Icarus Verilog, utilizing "Analog Step" data formatting in GTKWave to visually contrast the ZOH stair-step against the calculated slope.

2.1.3 Mixed-Signal Simulation (eSim & Ngspice)

Advanced Ngspice Plot Customization

To clearly contrast the harsh input data against the smoothed hardware output, the Ngspice `.control` script was configured to mathematically reconstruct the raw analog input staircase

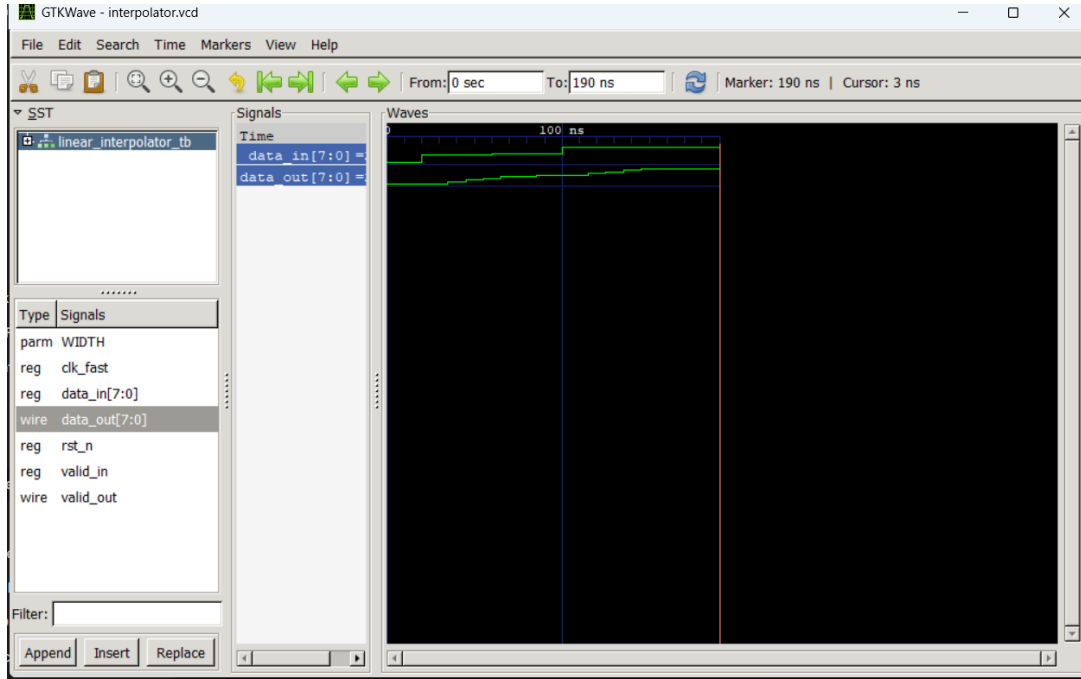


Figure 2.2: GTKWave Digital Simulation Waveforms confirming arithmetic shift calculations

and overlay it precisely against the DAC-reconstructed interpolated voltage on the same Y-axis grid.

* Ngspice Control Block for Analog Curve Visualization

```
.control
run
print allv > plot_data_v.txt
print alli > plot_data_i.txt
```

```
set xbrushwidth=3
set color0=white
set color1=black
```

```
* Reconstruct the raw analog input using pulse stimulus pins
let raw_input = (v(data_in_6)/4 + v(data_in_5)/8)
```

```
* Mathematically sum the 8 DAC pins into a single continuous analog waveform
let smoothed_out = (v(out_msb)*(128/255) + v(out_6)*(64/255) + v(out_5)*(32/255)
+ v(out_4)*(16/255) + v(out_3)*(8/255) + v(out_2)*(4/255) + v(out_1)*(2/255)
+ v(out_lsb)*(1/255))
```

```
* Plot them both together
plot smoothed_out raw_input
```

```
.endc
```

Interpretation of Output Waveforms

During initial mixed-signal verification, a hardware edge-case regarding Two's Complement arithmetic was documented. When applying stimulus to the Most Significant Bit (`data_in[7]`), the signed digital logic interpreted the input as a negative integer (-128), resulting in a severe analog glitch ($\sim 4.4V$) when processed by the unsigned eSim DAC bridge. To ensure flawless

continuous interpolation, the final stimulus was restricted to the lower 7 bits, keeping the sign bit securely grounded.

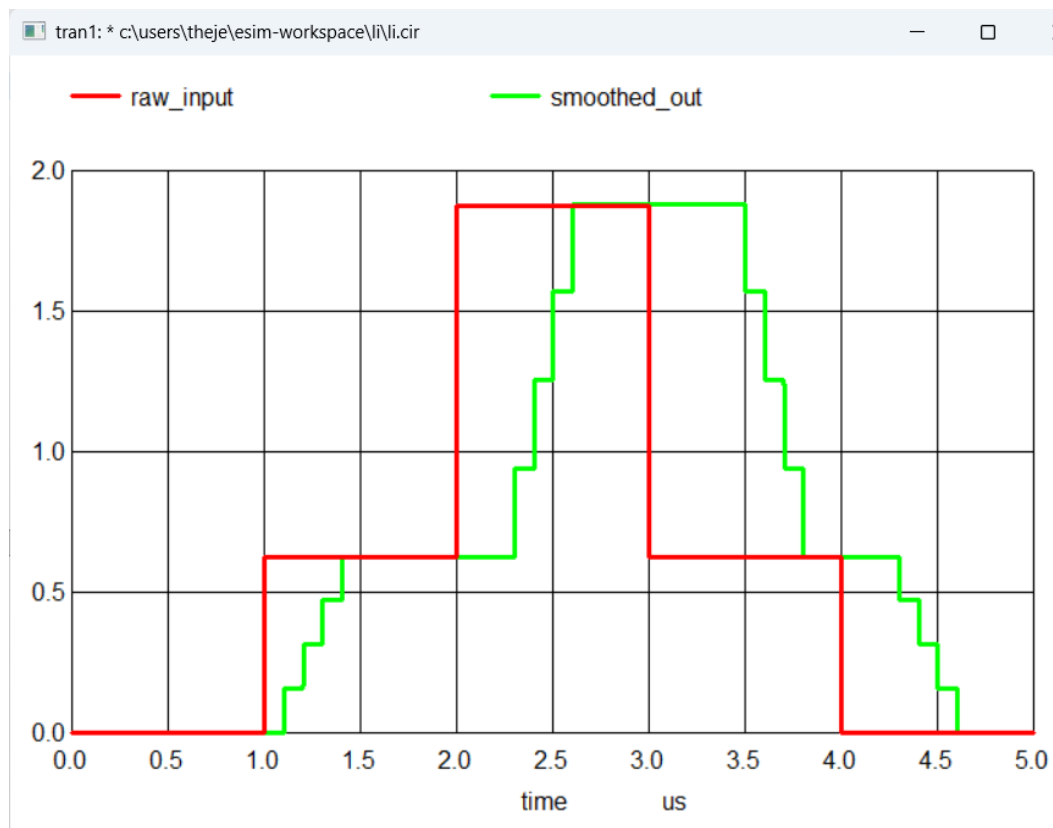


Figure 2.3: Ngspice Mixed-Signal Simulation contrasting ZOH steps (red) against the linearly interpolated slope (green)

As evidenced by the generated waveform data:

1. **Fractional Step Insertion:** Between every wide `valid_in` pulse, the fast master clock successfully calculates and inserts exactly three equidistant intermediate voltage points, peaking accurately at $\sim 1.88V$ during maximum stimulus.
2. **ZOH Mitigation:** While the reconstructed raw input signal jumps violently from level to level, the DAC output forms a continuous, mathematically calculated diagonal slope.
3. **Pipeline Stability:** The smooth descent back to 0V proves the structural efficacy of the arithmetic shift pipeline, verifying that the IP block safely translates digital data into the analog domain without boundary overflows or ZOH distortion.